

~\OneDrive\Desktop\digitsV2.py

```

1  import numpy as np
2
3  # storing the 5 patterns from the openTA
4
5  x1 = [[-1, -1, -1, -1, -1, -1, -1, -1, -1],
6         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
7         [-1, -1, 1, 1, 1, 1, 1, 1, -1],
8         [-1, 1, 1, 1, -1, -1, 1, 1, 1],
9         [-1, 1, 1, 1, -1, -1, 1, 1, 1],
10        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
11        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
12        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
13        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
14        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
15        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
16        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
17        [-1, 1, 1, 1, -1, -1, 1, 1, 1],
18        [-1, -1, 1, 1, 1, 1, 1, 1, -1],
19        [-1, -1, -1, 1, 1, 1, 1, -1, -1],
20        [-1, -1, -1, -1, -1, -1, -1, -1, -1]]
21
22  x2 = [[-1, -1, -1, 1, 1, 1, 1, -1, -1],
23         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
24         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
25         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
26         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
27         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
28         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
29         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
30         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
31         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
32         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
33         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
34         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
35         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
36         [-1, -1, -1, 1, 1, 1, 1, -1, -1],
37         [-1, -1, -1, 1, 1, 1, 1, -1, -1]]
38
39  x3 = [[1, 1, 1, 1, 1, 1, 1, 1, -1],
40         [1, 1, 1, 1, 1, 1, 1, 1, -1],
41         [-1, -1, -1, -1, -1, 1, 1, 1, -1],
42         [-1, -1, -1, -1, -1, 1, 1, 1, -1],
43         [-1, -1, -1, -1, -1, 1, 1, 1, -1],
44         [-1, -1, -1, -1, -1, 1, 1, 1, -1],
45         [-1, -1, -1, -1, -1, 1, 1, 1, -1],
46         [1, 1, 1, 1, 1, 1, 1, 1, -1],
47         [1, 1, 1, 1, 1, 1, 1, 1, -1],
48         [1, 1, 1, -1, -1, -1, -1, -1, -1],
49         [1, 1, 1, -1, -1, -1, -1, -1, -1],
50         [1, 1, 1, -1, -1, -1, -1, -1, -1],
51         [1, 1, 1, -1, -1, -1, -1, -1, -1]]

```

```

52     [1, 1, 1, -1, -1, -1, -1, -1, -1, -1],
53     [1, 1, 1, 1, 1, 1, 1, 1, -1, -1],
54     [1, 1, 1, 1, 1, 1, 1, 1, -1, -1]]
55
56 x4 = [[-1, -1, 1, 1, 1, 1, 1, 1, -1, -1],
57        [-1, -1, 1, 1, 1, 1, 1, 1, -1],
58        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
59        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
60        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
61        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
62        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
63        [-1, -1, 1, 1, 1, 1, 1, 1, -1, -1],
64        [-1, -1, 1, 1, 1, 1, 1, 1, -1, -1],
65        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
66        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
67        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
68        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
69        [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
70        [-1, -1, 1, 1, 1, 1, 1, 1, 1, -1],
71        [-1, -1, 1, 1, 1, 1, 1, 1, -1, -1]]
72
73 x5 = [[-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
74        [-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
75        [-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
76        [-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
77        [-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
78        [-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
79        [-1, 1, 1, -1, -1, -1, -1, 1, 1, -1],
80        [-1, 1, 1, 1, 1, 1, 1, 1, 1, -1],
81        [-1, 1, 1, 1, 1, 1, 1, 1, 1, -1],
82        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1],
83        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1],
84        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1],
85        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1],
86        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1],
87        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1],
88        [-1, -1, -1, -1, -1, -1, -1, 1, 1, -1]]
89
90 def matrix_to_vector(mat):
91     # convert matrix to vector using typewriter ordering
92     vec = np.array(mat).flatten()
93     return vec
94
95 def vector_to_matrix(vec):
96     # back to 16x10
97     mat = vec.reshape(16, 10)
98     return mat
99
100 def create_weight_matrix(patterns):
101     # implementing hebb's rule from eq 2.26
102     N = len(patterns[0]) # number of neurons
103     P = len(patterns)    # number of patterns
104
105     W = np.zeros((N, N)) # A matrix with the weights of all possible connections

```

```

106
107     # sum over all patterns
108     for mu in range(P):
109         for i in range(N):
110             for j in range(N):
111                 if i != j: # w_ii must be 0
112                     W[i, j] = W[i, j] + patterns[mu][i] * patterns[mu][j]
113
114     # normalize
115     W = W / N
116     return W
117
118 def sign_function(h):
119     # sgn(0) = 1 as specified
120     if h >= 0:
121         return 1
122     else:
123         return -1
124
125 def asynchronous_update(state, W):
126     N = len(state)
127     is_converged = False
128     iter = 0
129     max_iter = 1000 # just to be safe
130
131     while is_converged == False and iter < max_iter:
132         prev_state = state.copy()
133
134         # update neurons one by one (typewriter order)
135         for i in range(N):
136             local_field = 0
137             for j in range(N):
138                 local_field = local_field + W[i, j] * state[j]
139             state[i] = sign_function(local_field)
140
141         # check if we reached steady state
142         if np.array_equal(state, prev_state):
143             is_converged = True
144
145         iter = iter + 1
146
147     return state, is_converged
148
149 def classify_pattern(result, stored):
150     # check all stored patterns
151     for idx in range(len(stored)): # from 0 to 4
152         # normal pattern
153         if np.array_equal(result, stored[idx]):
154             return idx + 1
155         # inverted pattern
156         elif np.array_equal(result, -stored[idx]):
157             return -(idx + 1)
158
159     # no match found

```

```
160     return 6
161
162 if __name__ == "__main__":
163     print("Hopfield Network Exercise\n")
164
165     # prepare patterns
166     patterns = []
167     patterns.append(matrix_to_vector(x1))
168     patterns.append(matrix_to_vector(x2))
169     patterns.append(matrix_to_vector(x3))
170     patterns.append(matrix_to_vector(x4))
171     patterns.append(matrix_to_vector(x5))
172
173     # build weight matrix
174     W = create_weight_matrix(patterns)
175     print("Weight matrix created\n")
176
177     # test patterns from exercise
178     test_1 = [[1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
179              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
180              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
181              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
182              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
183              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
184              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
185              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
186              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
187              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
188              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
189              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
190              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
191              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
192              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1],
193              [1, -1, -1, 1, 1, -1, 1, -1, -1, -1]]
194
195     test_2 = [[-1, 1, 1, 1, 1, -1, 1, 1, -1, -1],
196              [-1, 1, 1, 1, 1, -1, 1, 1, -1, -1],
197              [1, -1, -1, -1, -1, -1, 1, 1, -1, -1],
198              [1, -1, -1, -1, -1, -1, 1, 1, -1, -1],
199              [1, -1, -1, -1, -1, -1, 1, 1, -1, -1],
200              [1, -1, -1, -1, -1, -1, 1, 1, -1, -1],
201              [1, -1, -1, -1, -1, -1, 1, 1, -1, -1],
202              [-1, 1, 1, 1, 1, -1, 1, 1, -1, -1],
203              [-1, 1, 1, 1, 1, -1, 1, 1, -1, -1],
204              [-1, 1, 1, -1, -1, 1, -1, -1, -1, -1],
205              [-1, 1, 1, -1, -1, 1, -1, -1, -1, -1],
206              [-1, 1, 1, -1, -1, 1, -1, -1, -1, -1],
207              [-1, 1, 1, -1, -1, 1, -1, -1, -1, -1],
208              [-1, 1, 1, -1, -1, 1, -1, -1, -1, -1],
209              [-1, 1, 1, 1, 1, -1, 1, 1, -1, -1],
210              [-1, 1, 1, 1, 1, -1, 1, 1, -1, -1]]
211
212     test_3 = [[1, 1, -1, -1, -1, -1, -1, -1, 1, 1],
213              [-1, -1, 1, 1, 1, 1, 1, 1, 1, -1],
```

```
214         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
215         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
216         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
217         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
218         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
219         [-1, -1, 1, 1, 1, 1, 1, 1, -1, -1],
220         [-1, -1, 1, 1, 1, 1, 1, 1, -1, -1],
221         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
222         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
223         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
224         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
225         [-1, -1, -1, -1, -1, -1, 1, 1, 1, -1],
226         [-1, -1, 1, 1, 1, 1, 1, 1, 1, -1],
227         [-1, -1, 1, 1, 1, 1, 1, 1, -1, -1]]
228
229     test_list = [test_1, test_2, test_3]
230
231     # run tests
232     for test_num, test_pat in enumerate(test_list, 1): # enumerate creates a tuple (index,
value)
233         print(f"Test Pattern {test_num}:")
234
235         test_vec = matrix_to_vector(test_pat)
236         result_vec, converged = asynchronous_update(test_vec.copy(), W)
237
238         if converged:
239             classification = classify_pattern(result_vec, patterns)
240
241             # format output as required
242             result_mat = vector_to_matrix(result_vec)
243             print("Final pattern:")
244             output = []
245             for row in result_mat:
246                 output.append([int(x) for x in row])
247             print(str(output).replace(" ", ""))
248
249             print(f"Classification: {classification}\n")
250         else:
251             print("Did not converge\n")
```